AdvR 08 – Functionals

map()

What Is a Functional?

A **functional** takes a function as input and returns a vector:

```
randomise <- function(f) f(runif(1e3))
randomise(mean)
```

```
#> [1] 0.50244
```

```
randomise(sum)
```

```
#> [1] 496.213
```

Functionals replace `for` loops with **intention-revealing** code. Each functional is tailored for a specific task.

Rule: if a functional fits, use it. If not, keep the `for` loop.

map() Basics

purrr::map() calls a function on each element, returns a **list**:

```
triple <- function(x) x * 3
map(1:3, triple)
```

```
#> [[1]]
#> [1] 3
#>
#> [[2]]
#> [1] 6
#> ... [output truncated]
```

Type-stable variants return atomic vectors:

```
map_dbl(1:3, triple)
```

```
#> [1] 3 6 9
```

```
map_chr(1:3, ~ paste0("Number: ", .x))
```

```
#> [1] "Number: 1" "Number: 2" "Number: 3"
```

```
map_lgl(1:3, ~ .x > 1)
```

```
#> [1] FALSE  TRUE  TRUE
```

Anonymous Functions and Shortcuts

Anonymous function (formula interface):

```
map_dbl(1:3, ~ .x ^ 2)
```

```
#> [1] 1 4 9
```

Extraction shortcuts — select elements by name, position, or both:

```
x <- list(list(a = 1, b = 2), list(a = 3, b = 4))
map_dbl(x, "a")        # by name
```

```
#> [1] 1 3
```

```
map_dbl(x, 2)           # by position
```

```
#> [1] 2 4
```

Behind the scenes, shortcuts use `as_mapper()` + `pluck()`.

Passing Arguments

Pass extra arguments via `...`:

```
x <- list(c(1, NA, 3), c(4, 5, 6))
map_dbl(x, mean, na.rm = TRUE)
```

```
#> [1] 2 5
```

Or via formula (safer — avoids partial matching):

```
map_dbl(x, ~ mean(.x, na.rm = TRUE))
```

```
#> [1] 2 5
```

Key difference: arguments in `...` are evaluated **once** and passed to every call. Formulas are re-evaluated each iteration.

Exercises (`map()`)

1. What is the difference?

```
map(1:3, ~ runif(2))   # (a)
map(1:3, runif(2))      # (b)
```

2. Use `map()` and the relevant string/numeric/logical variant to:
   a. Compute the mean of every column in `mtcars`
   b. Determine the type of each column in `nycflights13::flights`
   c. Compute the number of unique values in each column of `iris`

Solutions (`map()`) — 1

1. **(a)** `~ runif(2)` is a formula → function: each call generates a **new** pair of random numbers. Returns 3 different length-2 vectors.

(b) `runif(2)` is evaluated **once** (producing a numeric vector), then used as an extractor via `pluck()`. Returns `NULL`s.

Solutions (map()) — 2

❷

```
map_dbl(mtcars, mean)
```

```
#>        mpg        cyl       disp         hp       drat
#>  20.090625   6.187500 230.721875 146.687500   3.596563
#>         wt       qsec         vs         am       gear
#> ... [output truncated]
```

```
map_chr(mtcars, typeof)
```

```
#>      mpg      cyl     disp       hp     drat       wt
#> "double" "double" "double" "double" "double" "double"
#>     qsec       vs       am     gear     carb
#> ... [output truncated]
```

```
map_int(iris, ~ length(unique(.x)))
```

```
#> Sepal.Length  Sepal.Width Petal.Length  Petal.Width
#>           35           23           43           22
#>      Species
#> ... [output truncated]
```

Purrr Style

Chaining Functionals

`map()` returns a list → chain with `|>` or `%>%`:

```
mtcars |>
  split(mtcars$cyl) |>
  map(\(df) lm(mpg ~ wt, data = df)) |>
  map(summary) |>
  map_dbl("r.squared")
```

```
#>         4         6         8
#> 0.5086326 0.4645102 0.4229655
```

Pipeline pattern: data → split → fit models → extract summaries → pull metric.

Map Variants

Overview of Variants

Two dimensions: **(1) output type** and **(2) number of inputs**:

	List	Atomic	Same type	Side effect
1 arg	`map()`	`map_*()`	`modify()`	`walk()`
2 args	`map2()`	`map2_*()`	`modify2()`	`walk2()`
n args	`pmap()`	`pmap_*()`	—	`pwalk()`

Plus: `imap()` (iterates with index/name), `modify_if()` (conditional).

`modify()` — Same Type Out

`modify()` returns the **same type** as its input:

```
df <- data.frame(x = 1:3, y = 6:8)
modify(df, ~ .x * 2)
```

```
#>   x  y
#> 1 2 12
#> 2 4 14
#> 3 6 16
```

Compare: `map(df, ~ .x * 2)` returns a **list**, not a data frame.

map2() and pmap()

`map2()` iterates over two inputs in parallel:

```
map2_dbl(1:3, 4:6, ~ .x + .y)
```

```
#> [1] 5 7 9
```

`pmap()` takes a **list** of inputs (any number):

```
pmap_dbl(list(1:3, 4:6, 7:9), sum)
```

```
#> [1] 12 15 18
```

Useful with data frames (each row = one call):

```
params <- tibble::tibble(n = 1:3, min = c(0, 10, 100))
pmap(params, runif)
```

```
#> [[1]]
#> [1] 0.6499351
#>
#> ... [output truncated]
```

imap() — Iterate with Index

`imap(x, f)` is shorthand for `map2(x, names(x), f)` (or indices if unnamed):

```
imap_chr(mtcars[1:3], ~ paste(.y, "mean:", round(mean(.x), 1)))
```

```
#>                 mpg                 cyl
#>   "mpg mean: 20.1"    "cyl mean: 6.2"
#>                disp
#> ... [output truncated]
```

`walk()` — Side Effects Only

`walk()` calls a function for its **side effects**, returns input invisibly:

```
walk(1:3, ~ cat("Step", .x, "\n"))
```

```
#> Step 1
#> Step 2
#> Step 3
#> ... [output truncated]
```

Use `walk2()` for two inputs (e.g., writing multiple files):

```
walk2(data_frames, paths, write.csv)
```

Exercises (Variants)

1. Explain the results:

```
modify(mtcars, 1)
```

2. Rewrite this code to use `iwalk()`:

```
cyls <- split(mtcars, mtcars$cyl)
paths <- paste0("cyl-", names(cyls), ".csv")
walk2(cyls, paths, write.csv)
```

Solutions (Variants)

1. `modify(mtcars, 1)` extracts the **first element** of each column, returning a 1-row data frame (same type as input).
2.

```
mtcars |>
  split(mtcars$cyl) |>
  iwalk(~ write.csv(.x, paste0("cyl-", .y, ".csv")))
```

Reduce

reduce() and accumulate()

`reduce()` collapses a vector by successively applying a binary function:

```
reduce(1:4, `+`)   # ((1+2)+3)+4 = 10
```

```
#> [1] 10
```

Always supply `.init` for robustness (handles empty input):

```
reduce(1:4, `+`, .init = 0)
```

```
#> [1] 10
```

`accumulate()` keeps all intermediate results:

```
accumulate(1:4, `+`)
```

```
#> [1]  1  3  6 10
```

reduce() Use Case: Merging

```r
dfs <- list(
  data.frame(id = 1:2, a = c("x", "y")),
  data.frame(id = 1:2, b = c(10, 20)),
  data.frame(id = 1:2, c = c(TRUE, FALSE))
)
reduce(dfs, merge, by = "id")
```

```
#>   id a  b     c
#> 1  1 x 10  TRUE
#> 2  2 y 20 FALSE
```

Exercises (Reduce)

1. Explain how the result changes when you switch from `reduce()` to `accumulate()`.
2. Implement a `span()` function that, given a list `x` and a predicate `p`, returns the longest sequential run of elements where `p` is `TRUE`.

Solutions (Reduce)

1. `reduce()` returns only the final result; `accumulate()` returns a vector of **all intermediate results** (same length as input).

```
reduce(1:4, `*`)       # 24
```

```
#> [1] 24
```

```
accumulate(1:4, `*`)   # 1, 2, 6, 24
```

```
#> [1]  1  2  6 24
```

2. Use `rle()` on the logical vector to find the longest `TRUE` run:

```
span <- function(x, p) {
  idx <- unlist(map(x, p))
  rle_val <- rle(idx)
  good <- rle_val$values & rle_val$lengths == max(rle_val$lengths[rle_val$values])
  x[rep(good, rle_val$lengths)]
}
```

Predicate Functionals

A **predicate** returns a single `TRUE` or `FALSE`.
Three families:

Task	Function	Result
Any match?	`some(x, p)`	Single logical
All match?	`every(x, p)`	Single logical
First match	`detect(x, p)`	Value
First index	`detect_index(x, p)`	Integer
Keep matches	`keep(x, p)`	Subset
Drop matches	`discard(x, p)`	Subset

Predicate Functionals in Action

```
df <- data.frame(a = 1:3, b = c("x", "y", "z"), c = 4:6)
keep(df, is.numeric)
```

```
#>   a c
#> 1 1 4
#> 2 2 5
#> ... [output truncated]
```

```
some(df, is.character)
```

```
#> [1] TRUE
```

```
every(df, is.numeric)
```

```
#> [1] FALSE
```

```
detect(list(1, "a", 3), is.character)
```

```
#> [1] "a"
```

```
detect_index(list(1, "a", 3), is.character)
```

```
#> [1] 2
```

map_if() and modify_if()

Apply a function **only where a predicate is true**:

```
str(map_if(iris[1:3, ], is.numeric, mean))
```

```
#> List of 5
#>  $ Sepal.Length: num 4.9
#>  $ Sepal.Width : num 3.23
#>  $ Petal.Length: num 1.37
#>  $ Petal.Width : num 0.2
#> ... [output truncated]
```

`modify_if()` preserves the input type:

```
modify_if(iris[1:3, ], is.numeric, round, digits = 0)
```

```
#>   Sepal.Length Sepal.Width Petal.Length Petal.Width
#> 1            5           4            1           0
#> 2            5           3            1           0
#> 3            5           3            1           0
#>   Species
#> ... [output truncated]
```

Base R Functionals

apply() Family

`apply(X, MARGIN, FUN)` for matrices/arrays:

```
m <- matrix(1:12, nrow = 3)
apply(m, 1, sum)   # row sums
```

```
#> [1] 22 26 30
```

```
apply(m, 2, sum)   # col sums
```

```
#> [1]  6 15 24 33
```

Caution: never use `apply()` on data frames (coerces to matrix first).

Mathematical functionals (no purrr equivalent):

```
integrate(sin, 0, pi)$value
```

```
#> [1] 2
```

```
uniroot(\(x) x^2 - 4, c(0, 3))$root
```

```
#> [1] 2.000001
```